



01.01	Open the marketing site / app/page.tsx client component: SceneStack of 11 scroll scenes (Hero, Features, HowItWorks, Integrations, Industries, Testimonials, Pricing, FAQ, Contact, FinalCta, Footer) with Lenis smooth scroll, ScrollProgress and Section3D depth.	OWNER
01.02	Jump to a section (nav / anchors) Navigation + SmoothScroll: desktop anchor links scroll through Lenis; every scene wrapper id is unique so getElementById stays unambiguous.	OWNER
01.03	Arrive through a referral link ?ref=CODE Read via URLSearchParams AFTER mount so SSR and the first client render match (avoids a hydration mismatch); the code is threaded into the pricing CTA and the Paddle checkout custom_data. -> P3_ref	OWNER
01.04	Ask a question / request a feature POST /api/contact: honeypot field _hp, in-memory rate limit 3 per IP per minute, escaped HTML email through Resend to SUPPORT_EMAIL. POST /api/feature-request stores the idea.	OWNER
01.05	Read privacy / terms / pricing policy /privacy, /terms, /pricing-policy and /google-user-data are public on purpose: Paddle requires a pricing policy and Google requires a Limited Use disclosure page.	OWNER
01.06	Sign up or sign in Clerk modal from ClerkProvider in app/layout.tsx. The session cookie is what every protected route and API call is authorised by.	OWNER
01.07	Edge gate: Clerk middleware proxy.ts (Next 16's renamed middleware). Public allowlist: /, /privacy, /terms, /pricing-policy, /google-user-data, /api/webhooks/*, /api/cron/*, /api/health, /api/reviews/*, /api/pusher/*, /api/contact. Everything else calls auth.protect().	OUR CODE
01.08	Server guard on the dashboard shell app/dashboard/layout.tsx: auth() -> redirect('/') with no session, currentUser() for the display name, findBusinessByUserId() for the tenant, then isTrialExpired() and the minutes maths for the AI status pill.	OUR CODE
01.09	DashboardAccessGate picks one of four states No business doc -> OnboardingFlow. Paddle success flag -> PaddleSuccessWaiting. Trial finished -> TrialEndedScreen. Has a doc but not active -> Paywall. status === 'active' -> the real dashboard.	OUR CODE

02.01	Fill the onboarding wizard OnboardingFlow: business name, owner name, phone, business type, industry, hours, services and free-form notes.	OWNER
02.02	POST /api/business/onboard The Clerk user is read server-side for owner_email; the document is upserted keyed on business_id = Clerk userId, so re-submitting can never create a second tenant.	OUR CODE
02.03	businesses doc is created status 'pending', the six answers compiled into knowledge_base_text, a referral_code generated, counters zeroed, created_at stamped. This one document is the root every later feature keys off.	STORED
02.04	Gate immediately shows the Paywall No AI number, no calls and no integrations exist yet. Nothing is provisioned until money is collected, which is why trial and paid both flow through Paddle first.	OUR CODE

03.01	Choose a plan in the Paywall trial / standard / premium plus an optional referral code. Trial is 3 days and 50 minutes; standard is 200 min (\$299); premium is 500 min (\$399).	OWNER
03.02	POST /api/checkout/paddle The plan name is mapped to a PADDLE*_PRICE_ID environment variable on the SERVER, so a client cannot post a cheaper price id. custom_data carries clerk_user_id, plan, ref and business_name.	OUR CODE
03.03	Paddle transaction + overlay checkout Paddle creates the transaction and the browser opens it as an overlay. Card data never touches our servers.	PROVIDER
03.04	Pay On success Paddle redirects back with ?paddle=success and a transaction id.	OWNER
03.05	PaddleSuccessWaiting polls activation GET /api/business/status?transaction_id -> verifyPaidTransaction() calls Paddle for that transaction and refuses to activate unless custom_data.clerk_user_id matches the session AND status === 'completed'.	OUR CODE
03.06	Tenant becomes active status 'active', plan_type, minutes_limit (50/200/500), trial_started_at + trial_ends_at, paddle_customer_id and paddle_subscription_id stored.	STORED
03.07	POST /api/webhooks/paddle paddle-signature HMAC verified with a 5 minute replay window, then webhook_events gives idempotency. This path is the AUTHORITATIVE activation; the browser redirect is only a convenience.	PROVIDER
03.08	Webhook event router Event router: subscription.created activated -> full activation. subscription.updated -> reconcile the plan from the price id (limits reset only on a real change). canceled expired -> status 'cancelled' + a 30 day deletion date. purpose 'additional_minutes' -> \$inc minutes.	AUTOMATED
03.09	provisionTwilioNumber() Reuse the subaccount by its deterministic friendly name (a retry must not bill a second one) or create it, then buy a US toll-free number UNDER it with voiceUrl /api/webhooks/twilio/inbound and smsUrl /api/webhooks/twilio/sms-inbound. A failed purchase closes a fresh subaccount.	OUR CODE -> P10_sms
03.10	Subaccount + number stored twilio_subaccount_sid, twilio_number and the twilio_numbers array. A failure writes the sentinel PROVISIONING_FAILED so later code (numbers/add, health checks) can self-heal.	STORED
03.11	Referral bonus is credited The referrer is credited +40 minutes (standard) or +70 (premium) through an atomic { referral_applied_at: { \$exists: false } } claim, so two concurrent Paddle events cannot double-credit; self-referral is blocked. The founder gets a Telegram alert.	AUTOMATED -> P6_code
03.12	Buy extra minutes mid-cycle POST /api/checkout/paddle-minutes: 50 minute batches capped at 200 (standard) / 500 (premium), blocked on trial. The webhook credits them. Calls PAUSE at the limit - there is no overage billing.	OWNER -> P09_guard
03.13	Manage billing / cancel POST /api/billing/portal creates a Paddle customer portal session. Cancelling fires subscription.canceled -> 30 day grace, then the cleanup cron closes Twilio and deletes the tenant.	OWNER
03.14	Trial runs out Nothing to click: isTrialExpired() rejects calls and texts at the edge on the first request past trial_ends_at, and the hourly cron flips status to 'trial_expired' and emails the owner.	AUTOMATED -> P09_guard

04.01	Open /dashboard RSC reads the business doc and the latest 50 calls (withRetry so a flaky Astra call degrades to an empty list instead of a 500).	OWNER
04.02	Read the KPI cards DashboardCards + AnimatedNumber: calls, minutes, appointments, hot leads - all derived from the calls array the RSC already loaded, so no extra queries.	OWNER
04.03	Read the charts DashboardCharts: 7 day call volume + sentiment donut. Premium swaps in PremiumAnalytics: revenue, average job value, quote tracking, with CinematicChartCard.	OWNER -> P6_settings
04.04	Scan the recent calls list Five most recent calls with sentiment / lead / appointment / emergency badges and a deep link into /dashboard/calls.	OWNER
04.05	Check your AI numbers Rendered from business.twilio_numbers, filtering out the PROVISIONING_FAILED sentinel, so the owner never sees a fake number.	OWNER
04.06	Open the notification bell GET /api/notifications (50 latest + unread count), PATCH mark_read / mark_all_read, DELETE single. Types: hot_lead, emergency, appointment, missed_call, minutes_80 90 100, trial_expired.	OWNER
04.07	Watch the minutes counter MinutesCounter shows used/limit from the business doc; at the limit it points at Settings -> Billing to buy a pack or upgrade.	OWNER
04.08	Check the AI status pill AIStatusPill: active only when status === 'active' AND total_minutes_used < minutes_limit. This is the same condition the inbound call route enforces.	OWNER -> P09_guard
04.09	Work the setup checklist SetupProgressSidebar: 10 weighted items (profile, hours/services, knowledge, greeting, first call, SMS approved, calendar, review link, job value) each deep-linking to settings?focus=... and the widget hides itself at 100%.	OWNER -> P6_settings
04.10	Watch the live activity feed ActivityFeed subscribes to the Pusher channel private-business-{id}, event activity:new: incoming_call, ai_answered, understanding, appointment_confirmed, event_created, email_sent, sms_sent, hot_lead, emergency, minutes_80/90/100.	OWNER -> P11_pusher
04.11	Realtime channel authorisation POST /api/pusher/auth only signs the channel private-business-{userId} for the signed-in user, so one tenant can never subscribe to another tenant's feed.	OUR CODE -> P11_pusher
04.12	Navigate to Calls / Settings / Docs NavLinks (desktop sidebar) and MobileNav, plus the footer links to /dashboard/docs, /dashboard/support and the in-app legal pages.	OWNER

05.01	Open /dashboard/calls CallLogsTable renders the 50 most recent calls; CallRow is keyboard accessible and shows duration, source, sentiment, lead quality, appointment and emergency state.	OWNER
05.02	Read a transcript / summary Everything the analyst model extracted is stored on the call: transcript, 1-sentence summary, sentiment, lead_quality, appointment_date_time, quote_given / quote_amount, recording_url.	OWNER
05.03	Flag or unflag a call POST /api/calls/toggle-flag - flips is_flagged so the owner can keep a shortlist of leads worth chasing.	OWNER
05.04	Mark the job done POST /api/calls/mark-job { done }: sets job_status 'done' + job_done_at, appends to business.jobs_completed (last 20, used as AI context), then triggers the review request (email first, SMS fallback) and records review_status 'link_sent'. A second click returns 409 - never two review texts.	OWNER -> P10_review
05.05	Mark a no-show job_status 'no_show' - stops the 24h auto-completion cron from treating it as a completed job, so no review request goes out.	OWNER
05.06	Nothing clicked: the cron closes the loop The hourly job-done-followup finds booked jobs 24h in the past that are still 'pending', marks them 'auto_done' and sends the review request - so revenue-generating work is never silently forgotten.	AUTOMATED -> P11_cron

06.01	Open /dashboard/settings SettingsForm with ?focus=business knowledge greeting routing integrations sms billing - the same deep links the dashboard checklist uses.	OWNER
06.02	POST /api/business/update-settings One write path for the whole profile, and it validates before storing: business_timezone must round-trip through Intl, review_link must be https, zapier_webhook_url must be public https (SSRF guard).	OUR CODE
06.03	Edit name, type, service area, owner phone Feeds the AI's answers and the owner-SMS target, plus the dashboard header and every outgoing email signature.	OWNER
06.04	Set hours, services, exclusions, pricing rules, FAQs Compiled into knowledge_base_text - the exact string Retell injects into the voice prompt and the SMS brain, so editing here is editing what the AI is allowed to say.	OWNER -> P09_prompt
06.05	Set greeting tone, greeting text and AI name Sent as retell_llm_dynamic_variables so the AI introduces itself the way the owner wants on every call.	OWNER -> P09_prompt
06.06	Toggle routing rules forward_emergency, notify_hot_lead, sms_missed_call, email_followup, daily_summary, appointment_reminders, review_followup. The server re-enforces plan gating on write (premium-only keys forced false for lower plans), never trusting the client.	OWNER -> P09_fanout
06.07	Define what counts as an emergency emergency_definition is injected into the agent prompt and drives the transfer_call tool + the emergency webhook fan-out.	OWNER -> P09_emerg
06.08	Set the review link + average job value review_link is the destination of every review request; avg_job_value is premium-only and unlocks the revenue analytics cards.	OWNER -> P10_review
06.09	Add a Zapier / automation webhook Premium only. Stored only after isSafeWebhookUrl() proves it is https and not localhost / RFC1918, blocking SSRF from our own servers.	OWNER -> P09_fanout
06.10	Add or remove a phone number POST /api/numbers/add: premium only, max 3. Self-heals by provisioning first if the tenant never got a line, then buys another toll-free number UNDER the same subaccount with the webhooks already wired. POST /api/numbers/remove releases it.	OWNER
06.11	Share your referral code POST /api/business/referral-code returns the code generated at onboarding so a second business landing on /?ref=CODE credits this account with +40 / +70 minutes.	OWNER -> P3_ref
06.12	Open the SMS compliance tab GET /api/sms/compliance returns a 20+ field prefilled Toll-Free Verification form plus the current status (lazily refreshed from Twilio at most every 6h, so APPROVED shows up in-app without a cron).	OWNER
06.13	POST /api/sms/compliance Validates every field (URLs, emails, E.164 phone, enumeration values) then submitTollfreeVerification() to Twilio; the sync/reconcile step already runs through the recovery engine, so a Twilio auth failure is retried with subaccount scope instead of surfacing as a dead form.	OUR CODE -> P10_gate
06.14	Twilio reviews the verification pending -> approved rejected, with rejection reasons and the edit window mirrored back into sms_compliance. This single flag is what unlocks ALL outbound texting (reminders, review asks, missed-call follow-ups).	PROVIDER

07.01	Click Connect Google GET /api/integrations/google-calendar/auth: CSRF state in an httpOnly cookie, access_type=offline + prompt=consent to force a refresh token, scopes calendar.events + business.manage + userinfo.email.	OWNER
07.02	Google consent -> callback State cookie is compared and deleted, the code is exchanged, the refresh token and the account email (decoded from the id token) are stored, google_business_connected is set, and the user is redirected with ?connected=1.	PROVIDER
07.03	What the token unlocks Appointment -> Calendar events (all plans) and the hourly Google Reviews sync + AI review replies (paid plans). Token problems are healed by the recovery action GOOGLE_REFRESH_OAUTH_TOKEN.	STORED -> P09_calendar
07.04	Click Connect Facebook / Instagram GET /api/integrations/meta/auth: CSRF state cookie plus scopes pages_show_list, pages_messaging, instagram_manage_messages, pages_read_engagement, business_management.	OWNER
07.05	FB dialog -> callback Short-lived token -> long-lived token, /me/accounts, with a business-portfolio owned_pages fallback, then the primary page token, its Instagram business account and the page picture are stored.	PROVIDER
07.06	What the page token unlocks Meta DM auto-replies for premium tenants (Messenger + Instagram) and the sender identity shown in the settings UI.	STORED -> P10_meta
07.07	Disconnect an integration POST /api/integrations/google-calendar/disconnect and /api/integrations/meta/disconnect clear the stored tokens, which instantly stops calendar writes / DM replies and shows the card as disconnected.	OWNER

08.01	Open the chat widget ChatWidget is rendered for premium tenants in the dashboard layout; history comes from GET /api/chat/history and POST /api/chat/read clears the unread badge.	OWNER
08.02	Send a message or a photo POST /api/chat/send: 2000 char cap, 2MB photo cap, photos persisted to chat_photos, message id generated server-side so ordering survives retries.	OUR CODE
08.03	Staff is pinged on Telegram notifyAdminChat() sends an escaped message and returns the Telegram message_id, which is stored ON the message - that is the join key the reply webhook uses later.	PROVIDER
08.04	Support thread upserted + pushed live conversations { kind: 'support' } gets the message appended with \$push, read_by_admin_at reset to null, and notifyChatAdmins() fires chat:new on private-admin-chat so /admin/chat animates instantly.	STORED
08.05	Staff replies in /admin/chat POST /api/admin/chat/reply appends an owner-role message and just calls notifyChat() - the widget receives it in real time with no polling.	STAFF
08.06	Staff replies inside Telegram instead POST /api/webhooks/telegram: x-telegram-bot-api-secret-token must match, sender id must equal TELEGRAM_CHAT_ID, and the Telegram reply_to_message_id is resolved back to the conversation (direct lookup, then a bounded scan fallback) before the reply is appended and pushed to the widget.	STAFF

09.01	Customer calls your AI number Twilio hits POST /api/webhooks/twilio/inbound; verifyTwilioRequest() validates the X-Twilio-Signature so nobody can spoof a call into your minutes.	PROVIDER
09.02	Four guards before any AI runs RETELL_AGENT_ID missing -> loud Telegram alert instead of a silent outage. Business not found by number -> polite TwiML. isTrialExpired() -> in-app notification and an unavailable message. total_minutes_used >= minutes_limit -> missed-lead notification and an unavailable message. This is what enforces the no-overflow promise.	OUR CODE
09.03	registerPhoneCall + tenant prompt Retell is registered with the business_id metadata and dynamic variables: knowledge_base, greeting, greeting_tone, routing_rules, emergency_definition, owner_phone, customer_phone. Multi-tenant isolation is per-call, not per-deployment.	OUR CODE -> P6_hours
09.04	Voice prompt is assembled Everything set in Settings (knowledge base, greeting, tone, emergency definition, routing rules) plus the live business name / service area is bound into this single call - no shared mutable prompt anywhere.	STORED
09.05	TwiML dials Retell over SIP The route answers with Dial / Sip sip:{call_id}@sip.retellai.com. Retell cannot natively forward under this model, which is exactly why the emergency path below rewrites live TwiML instead.	PROVIDER
09.06	Emergency: the agent calls transfer_call POST /api/webhooks/retell/emergency (HMAC verified): an urgent SMS heads-up to the owner, a live dashboard toast, then the LIVE call's TwiML is rewritten to Dial the owner using the subaccount-scoped client (master creds get Twilio 20404). If the bridge fails, the founder is paged.	PROVIDER -> P11_telegram
09.07	Customer hangs up POST /api/webhooks/retell/call-ended: signature + a webhook_events in-flight window, then a call_id dedupe check, so a Retell retry can never double-count minutes or double-book a calendar event.	PROVIDER
09.08	One model call turns talk into data gpt-4o-mini in JSON mode returns summary, sentiment, lead_quality, appointment_booked, customer_email + name, is_emergency, appointment_date/time, duration, quote_given and quote_amount. A model failure drops to safe defaults instead of losing the call.	OUR CODE
09.09	Call row written + minutes debited \$inc total_minutes_used and total_calls_processed atomically (race-safe), while the full call document - transcript, analysis, job_status 'pending' - is stored for the dashboard.	STORED
09.10	Post-call automations fire in parallel Calendar event (with the timezone-corrected start time), Zapier webhook (premium), follow-up email to the customer, hot-lead alert, emergency alert, appointment notification, missed-call SMS under 10s, and 80/90/100% minute alerts. Each is independently try/caught so one provider outage cannot lose the others.	AUTOMATED
09.11	Google Calendar event created zonedTimeToUtc() converts the model's naive local time via business_timezone (naive parsing shifted US appointments 4-5h), midnight becomes 10:00 local, past dates roll forward, and the insert runs through executeWithRecovery so a 401/403 refreshes the token and retries once.	PROVIDER

10.01	Customer texts your number POST /api/webhooks/twilio/sms-inbound (signature verified). The same handler serves WhatsApp: the whatsapp: prefix is detected, stripped and re-applied on the way out, and the channel is recorded per message.	PROVIDER
10.02	TCPA opt-out runs FIRST classifyOptOutKeyword() is evaluated before the trial and throttle gates: STOP-family keywords are confirmed and the number is recorded in sms_optouts, START-family clears it. A customer must always be able to opt out, whatever state the account is in.	OUR CODE
10.03	Trial + abuse gates An expired trial gets one polite 'number is no longer active' reply. Otherwise a 25 messages-per-hour per-customer cap stops a runaway loop from burning AI credits.	OUR CODE
10.04	SMS brain with real tools gpt-4o-mini reads knowledge_base_text plus the thread history and can actually call book_appointment, confirm_appointment, reschedule_appointment and cancel_appointment - which write straight to the calls collection, so '1 / 2 / 3' reminder replies really do move the appointment.	OUR CODE
10.05	Reply goes through one choke point sendSmsReply() -> sendBusinessSms(), which enforces three things at once: TFV approved, not opted out, and the business's OWN subaccount as the sending scope (master-scope sends fail with Twilio 21660).	OUR CODE -> P6_sms_state
10.06	Outbound texting is gated Until Toll-Free Verification is approved, sends are logged and skipped rather than failed noisily - which is why the compliance tab is step 8 of the setup checklist.	STORED
10.07	Customer DMs your Facebook / Instagram POST /api/webhooks/meta/inbound: x-hub-signature-256 verified, webhook_events idempotency per sender + timestamp, business matched on meta_page_id, premium only.	PROVIDER -> P7_meta_use
10.08	3 second human buffer The handler waits 3s and re-reads the conversation: if a newer message arrived it aborts, so a customer typing three lines gets ONE answer instead of three - and messages are appended with \$push so nothing is overwritten.	OUR CODE
10.09	JSON brain + hard safety overrides One call returns intent, sentiment, confidence, leadStage, extracted name/phone, action and reply. Then deterministic overrides apply: confidence under 0.7 or angry/frustrated/urgent escalates, and an emergency intent always escalates - the model never gets the last word.	OUR CODE
10.10	Reply is sent through the Graph API The reply and the new conversation state are persisted, then posted to the page's send endpoint with the stored page token.	PROVIDER
10.11	Review request after the work is done sendReviewRequest(): email first if the caller gave an address, SMS fallback - and the SMS path is TFV-gated like everything else. Only ever sent when a review link or a Google profile is configured.	AUTOMATED
10.12	Appointment reminder with 1 / 2 / 3 The hourly cron emails (then texts) the customer in the business's own timezone and marks reminder_sent so it never repeats; the replies are handled by the booking tools above.	AUTOMATED

11.01	Hourly jobs (GitHub Actions) curl -H 'Authorization: Bearer CRON_SECRET' at appointment-reminders, job-done-followup, sync-reviews and expire-trials. Every cron route compares the secret with a timing-safe equal.	AUTOMATED
11.02	Daily + weekly jobs cleanup-cancelled at 00:00 (closes the Twilio subaccount FIRST and skips the delete if that fails, so a billed line can never be orphaned), daily-summary at 03:00 and weekly-summary on Monday mornings.	AUTOMATED
11.03	Google Reviews sync + AI replies For every connected paid business: accounts -> locations -> v4 reviews, then unreplied ones get a short SEO-aware reply generated by gpt-4o-mini and PUT back to Google. The same logic is exposed to n8n at POST /api/reviews/generate-reply behind an x-api-key check.	PROVIDER
11.04	Resend emails Daily/weekly summaries, trial expiry notice, minute-limit notice, customer follow-ups, appointment reminders and review requests. Every send is wrapped so email trouble never breaks a business flow.	PROVIDER
11.05	Pusher realtime Channels private-business-{id} (activity:new, chat:new) and private-admin-chat (chat:new). notifyActivity() / notifyChat() never throw: a Pusher outage degrades to no toasts, never a failed call.	PROVIDER
11.06	Telegram as the ops bridge Two jobs in one bot: the owner-support bridge (phase 8) and the system alarm channel - provisioning failures, a missing RETELL_AGENT_ID, an emergency transfer that could not be bridged, and every HIGH/CRITICAL recovery incident.	PROVIDER
11.07	n8n boss alerts Hot-lead and emergency events are POSTed to N8N_BOSS_ALERT_URL so the owner can fan them out to any other channel they run.	PROVIDER
11.08	Idempotency, retries and caps webhook_events is the dedupe ledger for Retell, Twilio, Paddle, Meta and Telegram; the contact form is rate limited (3/IP/min); per-customer SMS is capped; and withRetry wraps Astra reads so a flaky database degrades instead of crashing a page.	STORED

12.01	Open /admin app/admin/layout.tsx allows only privateMetadata.role === 'admin' or an ADMIN_EMAILS match (both backend-set signals) and redirects everyone else to /dashboard. publicMetadata is never a trust anchor because it is client-readable.	STAFF
12.02	Admin overview AdminDashboardClient: tenants, usage, minutes and margin per plan using lib/costing, plus the incident counters.	STAFF
12.03	Support inbox GET /api/admin/chat/list and /messages render the support threads; the admin layout shows an unread badge from a countDocuments() on read_by_admin_at: null.	STAFF -> sup_open
12.04	Incident console GET /api/admin/incidents with filters (free text, provider, severity, status, operation, date range) and a detail page per incident.	STAFF
12.05	Act on an incident POST /api/admin/incidents/[id] with retry / resolve / recover / dismiss / requires_action. Retry runs only a REGISTERED operation executor, refuses terminal statuses, and guards against two admins retrying the same incident with an inflight set.	STAFF
12.06	Recovery engine wraps provider calls executeWithRecovery(): normalize + redact, deterministic classification (Twilio codes, Google reasons, HTTP semantics - no AI involved), bounded retry, then a registry + policy-checked action. Maximum automatic recovery depth is 1.	OUR CODE
12.07	AI is a consultant, never an actor Only unknown or ambiguous errors reach gpt-4o-mini, only within a budget (ai-guard) and a fingerprint cache, and the answer must be an id from the backend registry - which is then re-checked by the same policy engine. When nothing is provably safe it fails CLOSED into an incident.	OUR CODE
12.08	Incident + audit trail Incidents aggregate by sha256 fingerprint with occurrence counts, a timeline, and every policy evaluation (checks[] with pass/fail reasons) kept as evidence for the admin console.	STORED -> P11_telegram
12.09	Every inbound edge verifies its sender Twilio signature, Retell HMAC with a 5 minute replay window, Paddle HMAC, Meta x-hub-signature-256 and the Telegram secret token - all compared with a timing-safe equal, and all fail closed.	OUR CODE
12.10	SSRF, spam and prompt-safety guards isSafeWebhookUrl() blocks private hosts on user-supplied webhook URLs, the contact form is honeypot + rate limited, and every AI layer (SMS, Meta, voice) has deterministic overrides above the model.	OUR CODE
12.11	Operational surfaces GET /api/health for uptime monitors, structured [recovery] JSON logs, and a once-per-process production warning if the recovery guards ever fall back to in-memory state because Astra credentials are missing.	OUR CODE